CS 6515: Graduate Algorithms

Lecture 14: Approximation Algorithms and Greedy Methods

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

Today's question. When exact optimization is computationally intractable, can we efficiently find a solution with a provable guarantee? We will use Vertex Cover as our central example and then study a short-path greedy algorithm for directed edge-disjoint paths.

1 Approximation guarantees

For an instance I, let $\mathrm{OPT}(I)$ denote the optimal value and (I) the value returned by an algorithm. The algorithm must return a *feasible* solution.

Definition (F). or $\alpha \geq 1$, an α-approximation satisfies

$$(I) \leq \alpha\,\mathrm{OPT}(I) \quad \text{(minimization)}, \qquad (I) \geq \frac{\mathrm{OPT}(I)}{\alpha} \quad \text{(maximization)}.$$

The guarantee holds for every instance, not just on average.

Approximation schemes. A PTAS accepts $\varepsilon > 0$ and produces a $(1+\varepsilon)$-approximation for minimization in time polynomial in the input size for each fixed ε. An FPTAS runs in time polynomial in both the input size and $1/\varepsilon$.

For maximization, the common convention is a guarantee of at least $(1-\varepsilon)\,\mathrm{OPT}$ (for $0 < \varepsilon < 1$). Recall the Knapsack FPTAS from earlier in the course.

Algorithmic theme. We will prove bounds by comparing our solution with a *certificate* or a structural bound on the unknown optimum.

2 Vertex Cover: two greedy algorithms

Given an undirected graph $G = (V, E)$, find a minimum-size set $C \subseteq V$ such that every edge has an endpoint in C. Write $n = |V|$ and $m = |E|$.

2.1 First greedy idea: choose the largest remaining degree

At each step, choose a vertex incident to the largest number of *uncovered* edges. Add it to the cover and remove all edges incident to it. Degrees are always measured in the graph of remaining edges.

Greedy-Degree-Cover(G)
$C \leftarrow \varnothing$; $E' \leftarrow E$
while $E' \neq \varnothing$:
  choose a vertex v incident to the maximum number of edges in E'
  $C \leftarrow C \cup \{v\}$
  remove from E' every edge incident to v
return C

Theorem (log approximation). For $m \geq 1$, Greedy-Degree-Cover returns a vertex cover of size at most $\mathrm{OPT}(1 + \ln m)$. Consequently it is an $O(\log n)$-approximation.

Proof. Feasibility is immediate: the algorithm stops only when every edge is covered. Fix an optimal vertex cover C^* with $|C^*| = \mathrm{OPT}$. Suppose $r > 0$ edges remain. The vertices of C^* still cover all r remaining edges. Counting incidences between these edges and vertices in C^*, some vertex of C^* is incident to at least r/OPT remaining edges. The greedy choice covers at least as many. Hence, writing r_t for the number of remaining edges after t choices,

$$r_{t+1} \leq r_t \left(1 - \frac{1}{\mathrm{OPT}}\right), \qquad r_t \leq m \left(1 - \frac{1}{\mathrm{OPT}}\right)^t \leq m e^{-t/\mathrm{OPT}}.$$

For $t > \mathrm{OPT} \ln m$, the right-hand side is less than 1, so the integer r_t must be zero. Thus the algorithm makes at most $\lfloor \mathrm{OPT} \ln m \rfloor + 1 \leq \mathrm{OPT}(1 + \ln m)$ choices. Since $m \leq n(n-1)/2 < n^2$ for a simple graph, $1 + \ln m = O(\log n)$. □

Interpretation. The number of uncovered edges decreases by a constant *fraction* in each step, but that fraction depends on $1/\mathrm{OPT}$. This is the familiar logarithmic greedy analysis from Set Cover, with edges as elements and vertices as sets.

2.2 A different greedy choice: build a maximal matching

The degree-based algorithm is very natural: we *cover* as many edges as possible in each step. Can we do better? One simple variation is to include two vertices at once by picking the edge with the most neighbors. We see next that even a relaxed version of this idea yields a better approximation.

A different greedy choice. Select an arbitrary uncovered edge (u, v) and take *both* endpoints. Delete all edges incident to either endpoint and repeat.

```
Greedy-Vertex-Cover(G)
C ← ∅; E' ← E
while E' ≠ ∅:
   choose any (u, v) ∈ E'
   C ← C ∪ {u, v}
   remove from E' every edge incident to u or v
return C
```

Theorem (2−approximation). Greedy-Vertex-Cover is a polynomial-time 2-approximation.

Proof. Let M be the edges chosen by the algorithm. No two edges of M share an endpoint, so M is a matching. At termination, every edge is incident to an endpoint of an edge of M, so C is a vertex cover. Every vertex cover must include at least one endpoint of *each* matching edge, and those edges are disjoint. Hence

$$|M| \leq \mathrm{OPT} \qquad \text{and} \qquad |C| = 2|M| \leq 2\,\mathrm{OPT}\,. \qquad \square$$

Terminology. The matching M is *maximal*: no further edge can be added to it. It need not be a *maximum* matching. Maximality suffices for feasibility; disjointness supplies the lower bound.

Tight example for this algorithm. On a single edge, the algorithm returns both endpoints, whereas an optimal cover contains one. Its factor 2 is attained.

Connection to duality. The matching M is a combinatorial lower-bound certificate: $|M| \leq \mathrm{OPT}$. Next lecture, an LP relaxation will provide another lower bound.

3 Directed edge-disjoint paths: greedy plus counting

Problem. Given a directed graph $G = (V, E)$ and terminal pairs (s_i, t_i) for $i = 1, \ldots, k$, route as many distinct pairs as possible. Choose at most one directed s_i–t_i path per pair, and require that every edge is used by at most one chosen path. Let $m = |E|$.

Unlike the single-source/single-sink version, this multicommodity optimization problem is computationally difficult. We seek a guarantee on the *number of routed pairs.*

Algorithm. Let $L = \max\{1, \lfloor\sqrt{m}\rfloor\}$ for $m \geq 1$. Repeatedly find any currently routable terminal pair with a directed path of length at most L (for example, use breadth-first search for shortest paths). Select such a path, remove its edges, and remove that pair from consideration. Stop when no such pair remains. Let A be the resulting set of paths.

Theorem (Disjoint Path Approximation). For $m \geq 1$, this algorithm returns at least $\mathrm{OPT}/(2\sqrt{m}+1)$ paths. In particular, it is an $O(\sqrt{m})$-approximation.

Proof plan. Fix an optimal collection O of edge-disjoint paths. Partition it into short paths (at most L edges) and long paths (more than L edges).

Long paths. Each long path uses at least $L+1$ edges. As the paths in O are edge-disjoint,

$$|O_{\text{long}}| \leq \frac{m}{L+1}.$$

Short paths. Consider a short optimal path Q for pair i. If greedy routes pair i, charge Q to the greedy path for that pair. Otherwise, because greedy has terminated, Q must share an edge with some selected greedy path; charge Q to one such path. The optimal paths are edge-disjoint, so at most L short optimal paths can be charged to a greedy path through shared edges. In addition, at most one path can be charged to it because it serves the same pair. Thus

$$|O_{\text{short}}| \leq (L+1)|A|.$$

If $\mathrm{OPT} > 0$, at least one terminal pair has a path of length at most L *or* all optimal paths are long. In the latter case A could be empty, so the short-path-only algorithm by itself does *not* give a multiplicative approximation. We fix this by also computing one arbitrary feasible path for one terminal pair, if any exists, and returning the larger of that one-path solution and A. Write $a \geq 1$ for the number of paths returned when $\mathrm{OPT} > 0$. Then

$$\mathrm{OPT} \leq (L+1)|A| + \frac{m}{L+1} \leq \left(L + 1 + \frac{m}{L+1}\right) a \leq (2\sqrt{m}+1)a.$$

(The last inequality uses $L = \lfloor\sqrt{m}\rfloor$ and $L + 1 > \sqrt{m}$, with $L + 1 \leq \sqrt{m} + 1$.) If $\mathrm{OPT} = 0$, the guarantee is trivial. □

Why the square root? The two losses compete: short-path charging costs about L per greedy path; the number of long optimum paths is at most m/L. Balance L and m/L by choosing $L \approx \sqrt{m}$.

Remark. The simple algorithm that repeatedly chooses the globally shortest available path also admits an $O(\sqrt{m})$ analysis. The cutoff version makes the short/long split explicit. We are *not* proving a hardness-of-approximation theorem today.

4 What to remember

1. A good approximation proof compares an efficiently constructed solution with a lower bound on OPT.

2. For Vertex Cover, a maximal matching certifies OPT $\geq |M|$ and gives a factor 2.
3. For edge-disjoint paths, a charging argument handles short optimal paths, and a counting argument handles long ones.
4. A bound tight for a particular algorithm is not automatically a hardness bound for all polynomial-time algorithms.

Next: Bridge to LP rounding. What if the lower-bound certificate is the optimum of a *linear relaxation*, rather than a matching or a counting argument?